UniStudio — Interview Preparation

A guide to presenting your project for the **LiveWall – AI Developer Internship**.
Part 1: how to talk about your project in the interview. Part 2: the technical foundations you should understand.

Prepared 9 June 2026 · Project: UniStudio (an AI product-photography studio). You don't need to memorise everything — understand the *why* behind each choice and you can answer almost any question.

PART 1 — Your Project, Tailored to LiveWall

1.1 Who LiveWall is (know your audience)

LiveWall is a **creative digital agency** (founded 2011, offices in Tilburg / Amsterdam / Antwerp). They don't sell one product — they build **interactive campaigns, activations, games, tools, apps, platforms and MVPs** for big brands. Named clients include **McDonald's, Rituals, KLM and 9292**.

Their self-description matters for your interview:

- "A hands-on team where **the people shaping the idea also build it**" — they value people who can both think and make.
- They build with **clean APIs, modular components, and a deployment process that lets teams ship fast**.
- For this internship, **using AI tools as part of the daily workflow is the standard**, not the exception. They want someone exploring "how new technologies, including AI, can contribute to unique digital experiences."
- The internship has two flavours: **Visual** (front-end, interactive web, creative tech like Three.js/shaders) and **Logical** (back-end systems, data, logic, APIs). *Your project shows both.*

> **The big picture:** LiveWall is not looking for a research scientist who trains neural networks. They want a **builder who integrates AI into real, shippable digital products** and isn't afraid to experiment. That is exactly what your project is. Lead with that framing.

1.2 Why UniStudio is a strong match (memorise this mapping)

What LiveWall wants	What UniStudio proves about you
Integrate AI into real digital products	You wired up **5 different AI providers** (Replicate, fal.ai, FASHN, a local Docker model, and Anthropic Claude) into one working app that generates images and video.
Build clean APIs & modular components	Your app has **38 API routes** and ~18 reusable modules, each self-contained. You enforce that logic lives in shared modules and is never duplicated.
Think and make in the same place	You designed the concept (e-commerce photo automation), the architecture, *and* built and deployed it yourself.
Ship fast / good deploy process	It **auto-deploys to Vercel** on every push to main. Live at a public URL.
Use AI tools in the daily workflow	You built this **using an AI coding assistant** — you already work the way they work. (See 1.6.)
Visual track (front-end / interactive)	React 19 + a **Fabric.js canvas editor**, drag-and-drop uploads, live before/after compare, toasts, responsive UI.

What LiveWall wants	What UniStudio proves about you
Logical track (back-end / systems)	Server-side orchestration, async job queues with polling, an optional database, per-operation cost tracking, and an AI agent that plans and runs multi-step pipelines.
Solve a real problem	It replaces several paid SaaS tools and turns raw product photos into store-ready images/video for a real catalogue of **486 products**.

1.3 Your 30-second pitch (say it out loud until it's natural)

> “I built **UniStudio**, a web app that automates e-commerce product photography using AI. A user uploads a raw product photo, and the app can remove the background, generate a new scene, put the garment on an AI-generated model, add shadows, upscale it, and even create a short promo video — all in one place. Technically it's a **Next.js and TypeScript full-stack app** that acts as an **orchestrator**: it doesn't run the AI models itself, it wraps several external AI providers behind one clean, modular API. It's deployed on Vercel and built around an ‘AI agent’ that analyses an image, plans the steps, and runs the independent ones in parallel.”

1.4 Your 2-minute version (Problem → Idea → How → Hard parts → Result)

Problem. A clothing/jewelry brand needs dozens of clean, consistent product photos and short videos. Doing it manually, or paying for 4–5 separate SaaS tools, is slow and expensive.

Idea. One app that does the whole pipeline, paying only per AI call, with the cheap or free operations done locally.

How. A Next.js App-Router project in TypeScript. The front-end is React with a canvas editor. The back-end is a set of API routes; each one validates the request, picks the right AI provider for that product type, calls it, and returns a result URL. Client state lives in Zustand stores; an optional PostgreSQL database (via Prisma) saves history and the cost of each job.

Hard parts I'm proud of. (1) The **orchestration**: providers behave differently — some synchronous, others an async queue you submit to and then poll. I wrote a small client for each. (2) The **AI agent**: it uses Claude vision to analyse the photo, Claude to plan the steps, then executes independent steps in parallel with `Promise.all` to save time. (3) A strict **no-duplication architecture** — exactly three product pipelines, shared logic in modules, so the codebase stays clean at ~37,000 lines.

Result. A deployed app that turns a raw photo into store-ready assets, covering a 486-product catalogue, with live cost tracking.

1.5 Their application asks for “a project you're proud of” + 3 bullets on why you fit

LiveWall's application is informal: CV/LinkedIn, a link to a project, and **3 bullets (or a short voice/video)** on why you fit. Use these:

- **I build, I don't just plan.** I took UniStudio from idea to a deployed full-stack AI app on Vercel — front-end, back-end APIs, database, and AI integration, all by myself.
- **I already work AI-first.** I integrate generative image/video models and LLMs into real products, and I build *with* AI tooling daily — the exact workflow your internship is built around.
- **I care about clean, modular systems.** My project enforces shared modules and zero duplicated logic, with clean APIs and a fast deploy pipeline — the way you describe building at LiveWall.

1.6 How to talk honestly about building with an AI assistant

> You used an AI assistant to build much of this. **Do not hide it** — LiveWall explicitly values AI in the daily workflow. But you must **own and understand** the result. Frame it like a senior engineer who uses tools.

"I built this working closely with an AI coding assistant — exactly how this internship describes working. My role was the **architect and decision-maker**: I decided the three-pipeline structure, chose the providers, defined the data flow, debugged the integration problems, and reviewed every change. The AI accelerated the typing; I owned the design and I understand how every part works."

Then **prove it** by being able to explain any piece (that's what Part 2 is for). The danger is not "you used AI" — it's not being able to explain your own project. Part 2 closes that gap.

1.7 Likely interview questions & strong answers

"Walk me through the architecture."

Three layers, same pattern everywhere: **UI** (a React panel + a hook that calls `fetch`) → **API route** on the server (validates, runs the processing logic) → **provider client** (talks to the external AI service). The result URL comes back, optionally gets saved to the database, and updates client state. "Once you understand one feature, you understand all of them."

"Why Next.js?"

Because it gives me **front-end and back-end in one codebase and one deploy**. Pages are React; the `app/api/*/route.ts` files are my back-end endpoints. It deploys to Vercel with zero config, which matched my need to ship fast.

"Where's the actual AI? Did you train models?"

I don't train models — I **integrate** them. The intelligence lives in external providers (Flux, Kolors, Wan/Kling video models, and Claude for vision and planning). My value is the **orchestration**: validation, choosing the right model per product type, chaining steps, handling async jobs, error handling, and cost control. That's the realistic, production way to ship AI features.

"What was the hardest bug / problem?"

Two options: (1) Providers return results differently — some synchronous, some an async queue I had to **submit then poll**; I had to handle timeouts (Vercel kills a request at 60s, so slow video routes get a 300s limit). (2) A subtle one: API keys from the environment had a trailing newline causing **401 errors**, so every provider client calls `.trim()` on the key.

"How does front-end state work?"

I use **Zustand**, a lightweight state library. Six stores; three persist to `localStorage` so work survives a refresh. One detail: the brand store *excludes* base64 image blobs from what it saves, because localStorage has a size limit — otherwise it crashes.

"How do you keep a 37k-line codebase clean?"

A strict rule: **exactly three product pipelines**, never a fourth, and **no duplicated logic** — shared behaviour lives in modules that pipelines call. If a pipeline swaps a provider, the module and docs are updated in the *same commit*.

"Tell me about the AI agent."

A 5-step engine: upload → analyse the image (Claude vision) → plan the steps (Claude) → user edits the plan → execute. The clever bit: independent steps (e.g. removing the background and generating a model) run **in parallel**, saving 25–35 seconds per photo. It also respects a **budget tier** — free, economy, or premium — and picks cheaper or better providers accordingly.

"What would you improve / what are the limits?"

Honest answer scores points: no authentication (single-user by design), no automated end-to-end tests for every provider, and provider costs vary. Next steps: auth + multi-user, a proper job queue/worker for long video jobs instead of in-request polling, and caching repeated AI calls.

“Why should we take you?”

“Because I already do the thing your internship is about: I ship real digital products that integrate AI, I work AI-first, and I care about clean modular code. Give me a brief and I'll build something, not just talk about it.”

1.8 Questions YOU should ask them

- “For the AI Developer role, is it more about integrating AI into client campaigns, or building internal tools and prototypes?”
- “What does a typical project timeline look like — how fast do you go from idea to shipped?”
- “Which AI tools and models is the team using day to day right now?”
- “Will I work mostly solo on a piece, or pair with designers and other developers?”
- “What does a successful internship look like to you after a few months?”

1.9 Quick fact sheet to recall under pressure

Thing	Number / fact
What it is	AI product-photography studio (web app)
Stack	Next.js 16, React 19, TypeScript, Zustand, Prisma + PostgreSQL, Tailwind, Sharp, Fabric.js
Size	~37,000 lines, 38 API routes, ~18 modules, 6 stores, 7 DB models
AI providers	Replicate, fal.ai, FASHN, local Docker model, Anthropic Claude
Core idea	An *orchestrator* — wraps external AI, doesn't run its own
Deploy	Auto-deploys to Vercel on push to main
Flagship feature	AI agent: analyse → plan → parallel execution, with budget tiers

PART 2 — Technical Foundations (the *why* behind every choice)

For each technology: **what it is**, a little **syntax**, and **why you chose it**. Understand these and you can answer almost any “how does it work” question. Don't memorise — understand.

2.1 The mental model of the whole app

```
Browser (UI)                    Server (Next.js API)        Outside world
-----------                     --------------------        -------------
React page / panel              route.ts handler            External AI provider
  -> hook (manages state)         -> validate input         (Replicate / fal / Claude)
    -> fetch('/api/bg-remove')    -> processing logic
                                  -> call provider client --> AI runs, returns URL
  update Zustand store  <-------- send JSON response <-------- result
```

Every feature is this same flow. Saying that shows you see the *pattern*, not just the code.

2.2 Next.js + the App Router

What: Next.js is a framework built on React. The App Router means your folder structure *is* your routing: a folder with a `page.tsx` becomes a web page; a folder with a `route.ts` becomes an API endpoint.

```
src/app/editor/page.tsx        ->  the web page at /editor
src/app/api/bg-remove/route.ts ->  the API endpoint at /api/bg-remove
```

Syntax of an API route:

```
// a POST endpoint
export async function POST(request: Request) {
  const body = await request.json();        // read the incoming data
  // ...do the work...
  return Response.json({ url: resultUrl });  // send JSON back
}
```

Server vs Client components: by default components render on the server (fast, no secrets leak). A file that needs interactivity (clicks, state) starts with the directive `"use client"` at the top.

Why you chose it: one codebase for front *and* back end, great Vercel deployment, and file-based routing keeps a big project organised.

2.3 React + components

What: React builds UI out of **components** — reusable functions that return markup (JSX). The UI is a function of **state**: when state changes, React re-renders.

```
function UploadButton({ label }) {         // props = inputs to a component
  const [busy, setBusy] = useState(false);  // state = data that can change
  return <button onClick={() => setBusy(true)}>{label}</button>;
}
```

JSX is HTML-like syntax inside JavaScript. **Props** are inputs from a parent. **Hooks** are special functions starting with `use`:

- `useState` — hold a piece of changing data.
- `useEffect` — run side-effects (e.g. cleanup) when something changes.
- Custom hooks (yours: `useImageProcessing`, `useAgentPipeline`) — bundle reusable logic.

Why: the component model lets you reuse UI pieces (your ~18 module panels share buttons, dropzones, sliders) and keeps the interface predictable.

2.4 TypeScript

What: JavaScript with **types**. You declare the shape of your data and the compiler catches mistakes *before* running.

```
type TryOnRequest = {
  garmentUrl: string;
  modelUrl: string;
  provider: "kolors" | "fashn" | "idm-vton";  // only these 3 allowed
};
```

Why: in a 37k-line app with many providers, types are documentation that can't go stale and they prevent "undefined is not a function" bugs. "Types let me refactor confidently."

2.5 State management with Zustand

What: a small library for **global state** — data many components share (gallery history, brand colours, video settings).

```
const useGalleryStore = create((set) => ({
  images: [],
  addImage: (img) => set((s) => ({ images: [...s.images, img] })),
}));
// in a component:
const images = useGalleryStore((s) => s.images);
```

Why Zustand over Redux: far less boilerplate, no providers wrapping the app, and it persists to localStorage easily.

Smart detail to mention: the brand store uses `partialize` to keep base64 image data *out* of localStorage, because the browser caps it at ~5 MB and saving big blobs would crash it.

2.6 API routes, REST and fetch

What: the front-end and back-end talk over HTTP. The browser sends a request with `fetch`; the route handler responds with JSON.

```
// front-end
const res = await fetch('/api/bg-remove', {
  method: 'POST',
  body: JSON.stringify({ imageUrl }),
});
const data = await res.json();   // { url: "..." }
```

Why this split: secrets (API keys) and heavy work stay on the **server** — never exposed to the browser. The client only knows your own endpoints, not the providers.

2.7 The orchestrator concept + async patterns (your favourite topic)

What: your back-end's real job is to call external AI and manage the waiting. Two patterns:

- **Synchronous** (Replicate): call, wait, get the result.
- **Async queue** (fal.ai / FASHN): *submit* a job, get an id, then **poll** ("is it done yet?") until the result is ready.

```
// async/await: pause until the promise resolves, without freezing everything
const job = await fal.submit(...);          // 1. submit
while (!done) {
  const status = await fal.status(job.id);  // 2. poll
  done = status.completed;
}
```

Parallel execution — the trick in your AI agent. Independent steps run at the same time:

```
// run two slow steps together instead of one-after-another
const [noBg, model] = await Promise.all([
  removeBackground(image),
  createModel(prompt),
]);   // saves 25-35 seconds
```

Why it matters: this is the difference between an app that feels slow and one that feels fast, and it shows you understand promises, not just `await` on one line.

2.8 Database: Prisma + PostgreSQL (and why it's optional)

What: Prisma is an **ORM** — you describe your tables in a schema, and it generates type-safe functions so you write JavaScript instead of raw SQL.

```
// schema describes a table
model ProcessingJob { id String @id; operation String; cost Float }
// query it in code
await prisma.processingJob.create({ data: { operation: "bg-remove", cost: 0.004 } });
```

The clever design choice: the database is **optional**. Every DB call is "null-guarded" — if `DATABASE_URL` isn't set, the app skips saving and keeps working. That's a mature decision interviewers like.

Why: to store history and the cost of each job — without forcing every user to set up a database.

2.9 Styling: Tailwind CSS

What: utility-first CSS — you style by composing small classes directly in the markup.

```
<button class="bg-blue-600 text-white px-4 py-2 rounded">Save</button>
```

Why: fast to build, consistent spacing/colours, no jumping between CSS files. Good for shipping quickly — which matches LiveWall's speed.

2.10 Free compute: Sharp (server) & WASM (browser)

What: not everything needs a paid AI call. **Sharp** is a fast image library that runs on your server for colour/brightness/shadows — **free**. For background removal, you can run a model **in the browser** via WebAssembly — also free, and the image never leaves the user's machine.

Why mention it: it shows **cost-awareness and engineering judgement** — you only spend money on AI when it's genuinely needed. "Free where possible, paid where it counts."

2.11 The canvas editor: Fabric.js

What: Fabric.js sits on top of the HTML `<canvas>` and gives you objects you can move, scale, layer, and add text to — like a mini Photoshop. This is your "Smart Editor."

Why relevant to LiveWall: this is exactly the "Visual / creative technology" side of their internship. If they lean visual, talk about this.

2.12 Environment variables & secrets

What: API keys live in environment variables (`process.env.REPLICATE_API_TOKEN`), never in code, never sent to the browser.

```
const key = process.env.FAL_KEY?.trim();   // .trim() removes a trailing newline
```

The war story: keys pulled from Vercel had a hidden newline at the end, making the provider reject them with 401. The fix was `.trim()` in every client. Small, real, and shows you debug from symptom to root cause.

2.13 Deployment, Git & Vercel

What: you commit with Git; pushing to the `main` branch triggers Vercel to build and deploy automatically to a public URL. Slow routes (video, avatars) get a higher time limit (`maxDuration: 300`) in `vercel.json` because Vercel cuts requests at 60s by default.

Why it matters to them: “push updates without slowing down” is literally how LiveWall describes their workflow. You already live it.

2.14 Mini-glossary (terms they might drop)

Term	Plain meaning
API	A defined way for two programs to talk over HTTP.
REST endpoint	A URL you send requests to (GET to read, POST to send data).
Async / await / Promise	Tools to wait for slow work (a network call) without freezing the app.
Component	A reusable, self-contained piece of UI.
State	Data that changes over time and drives what the UI shows.
ORM	A tool to query a database with code instead of raw SQL.
LLM	Large Language Model (e.g. Claude) — you use it for vision analysis and planning.
Generative model	An AI that creates new images/video (Flux, Kolors, Kling).
Orchestration	Coordinating multiple services/steps into one workflow — your app's core job.
Polling	Repeatedly asking “is it done yet?” until an async job finishes.
WASM	WebAssembly — near-native code running in the browser (your free bg-removal).
Environment variable	A secret/config value kept outside the code.
CI/CD	Automatic build & deploy when you push code (your Vercel setup).

Final advice: Speak in terms of *problems and decisions*, not just tool names. “I chose X *because* Y” beats “I used X.” Whenever you can, point to a real moment: the 401 newline bug, the parallel-execution speed-up, the optional database. Those concrete stories make an intern sound like a builder. Good luck.

— Want this in Spanish or Dutch, or as slides instead of a document? Just ask.